

UM PROJETO. TODAS AS CAMADAS.

CyberShield

Da interface ao servidor.

Um guia pratico de desenvolvimento web para quem ja programa e quer entender o caminho completo de uma aplicacao.

HTML + CSS + JavaScript / Node.js + Express / SQLite

Interface em ingles. Explicacoes em portugues. Exemplos retirados dos arquivos reais deste projeto.

Este material descreve uma aplicacao local de estudo. Nao e um servico de seguranca em operacao nem uma certificacao de prontidao para producao.

Seu mapa de leitura

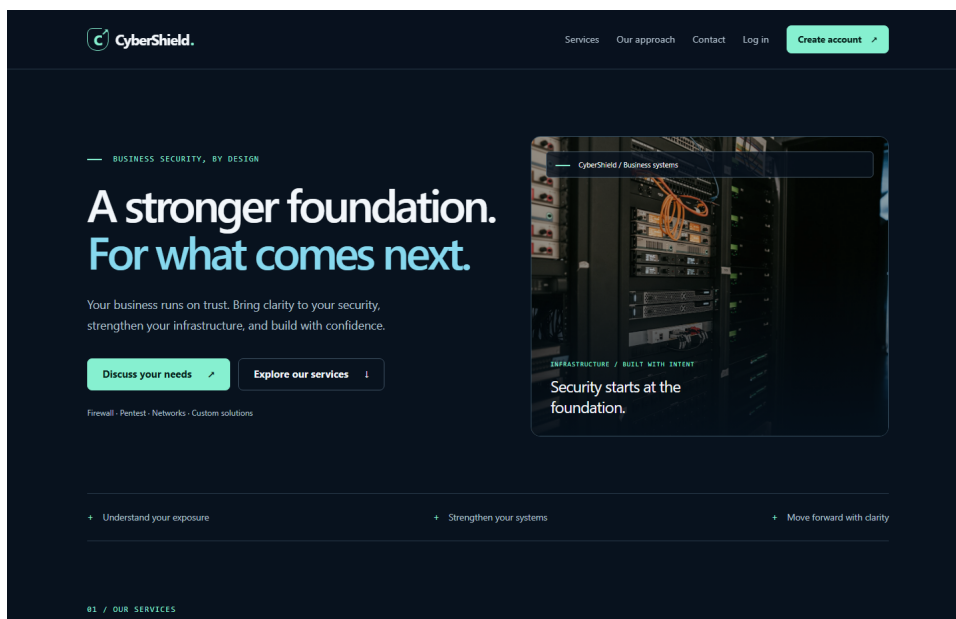
- 01 / 01. O que você construiu
- 02 / 02. A arquitetura e a fronteira de confiança
- 03 / 03. Pastas, execução e dependências
- 04 / 04. HTML, DOM e CSS na versão final
- 05 / 05. JavaScript no navegador: módulos e eventos
- 06 / 06. HTTP: o contrato entre os processos
- 07 / 07. Fetch e o auxiliar api.js
- 08 / 08. O ciclo do cadastro, ponta a ponta
- 09 / 09. Express: rotas, middlewares e ordem
- 10 / 10. Validação: experiência no cliente, autoridade no servidor
- 11 / 11. SQLite: persistência e relações
- 12 / 12. Senhas: hashing não é criptografia reversível
- 13 / 13. Sessão, cookie e logout
- 14 / 14. CSRF, XSS e camadas de defesa
- 15 / 15. Dashboard, perfil e pedidos de serviço
- 16 / 16. Testes: observar o comportamento, não só a sintaxe
- 17 / 17. DevTools e investigação de problemas
- 18 / 18. Limites antes de publicar e próximos passos
- 19 / 19. Exercícios com critérios e pistas
- 20 / 20. Referências e mapa final de leitura

Leia primeiro os capítulos de arquitetura e HTTP. Depois acompanhe um cadastro no DevTools e no banco. Use os exercícios para transformar leitura em prática.

01. O que você construiu

O CyberShield começou como uma landing page: o HTML descrevia as seções e o CSS definia sua aparência. Agora existe uma aplicação full stack: o navegador exibe a interface, o servidor aplica as regras e o banco mantém os dados depois que o processo termina.

Este guia assume que você já programa em Rust, C ou Python. Não vamos reaprender variáveis e condicionais. O foco é entender as fronteiras da web: DOM, eventos, HTTP, processos, persistência e identidade do usuário. Os exemplos numerados são extraídos dos arquivos reais, não de uma versão fictícia do projeto.



O escopo entregue

- Página inicial responsiva, com serviços e formulário de contato.
- Cadastro e login com verificação de senha no servidor.
- Cookie de sessão, logout e proteção da área do usuário.
- Perfil editável e histórico dos pedidos associados à conta.
- SQLite persistente, validação, proteção CSRF e testes automatizados.

É uma aplicação local de aprendizado. Não há um firewall real instalado, monitoramento de rede, equipe de suporte ou prestação de serviços ativada ao criar uma conta. O formulário salva o pedido no banco; não manda email. O dashboard não inventa gráficos de ataques nem indicadores de infraestrutura.

Como estudar sem decorar

Abra o projeto ao lado deste PDF. Acompanhe uma ação completa: clique, evento, requisição, rota, SQL, resposta, DOM. Depois mude uma regra pequena, escreva um teste e observe a consequência. O objetivo não é memorizar nomes de arquivos: é conseguir localizar a responsabilidade de cada camada.

02. A arquitetura e a fronteira de confiança



O navegador nunca recebe acesso direto ao arquivo do banco.

Somente public/ e servido. backend/ e docs/ ficam fora da area publica.

O frontend e o backend usam JavaScript, mas não compartilham automaticamente memória nem permissões. O navegador oferece document, eventos e Fetch. O Node oferece módulos do sistema, acesso a arquivos e o processo HTTP. O que acontece entre os dois são mensagens, não chamadas diretas a funções do outro processo.

O navegador envia um texto JSON. O servidor lê esse texto, transforma em objeto, valida os campos e decide se pode alterar o banco. Depois responde com status, cabeçalhos e outro texto JSON. O cliente interpreta a resposta e atualiza a interface.

Quem pode ser considerado confiável?

O visitante controla o navegador. Pode modificar o DOM, apagar required, executar código no Console ou enviar uma requisição sem usar seu site. Portanto, qualquer dado do cliente precisa passar pela validação do servidor.

A regra de propriedade também pertence ao backend. Para mostrar pedidos, não aceitamos um userId informado pelo formulário. O servidor obtém a identidade a partir da sessão validada e usa esse ID na consulta. Alterar um campo no navegador não deve dar acesso a pedidos de outro usuário.

Por que não migramos para React agora?

A aplicação é pequena e tem quatro páginas. HTML, CSS e módulos JS permitem enxergar o fluxo sem adicionar compilação, JSX e outras convenções. React poderia facilitar componentes e estados mais complexos, mas não substituiria HTTP, autenticação ou SQL. Next.js também não elimina essas fronteiras: muda a organização e oferece recursos de execução no servidor.

Exercício: explique, sem olhar o código, por que document.querySelector funciona no navegador, mas não é a maneira de acessar um usuário no SQLite.

03. Pastas, execução e dependências

```
cybershield-website-main/
public/
  index.html      pagina inicial e contato
  login.html      entrada na conta
  register.html   criacao da conta
  dashboard.html  perfil e pedidos
  css/style.css   design compartilhado
  js/api.js       Fetch, formularios e auxiliares
  js/main.js      comportamento da pagina inicial
  js/login.js     envio de login
  js/register.js  envio de cadastro
  js/dashboard.js perfil e historico
  img/           imagem local
backend/
  server.cjs      inicia e encerra o processo HTTP
  app.cjs         middlewares e rotas
  database.cjs    conexao e esquema SQLite
  passwords.cjs   hashing e verificacao
  sessions.cjs    cookies, identidade e CSRF
  validation.cjs  regras dos dados recebidos
  rate-limit.cjs  limitacao de requisicoes
  data/          banco gerado em execucao
  tests/         testes da API
scripts/e2e.cjs  testes reais no navegador
docs/           este manual, imagens e codigo original
```

Só `public/` é entregue como diretório estático pelo Express. O banco, o código do servidor e este PDF não devem ser expostos por esse mecanismo. Os HTMLs antigos foram preservados em `docs/original/` para comparação; não são a interface ativa.

Como iniciar

Use Node 24.13 ou superior. Na raiz do projeto:

```
npm install
npm run setup
npm start
```

Abra `http://127.0.0.1:3000/`. Para editar e reiniciar o servidor automaticamente, use `npm run dev` no lugar de `npm start`. O terminal precisa permanecer aberto. Um servidor é um processo em execução, não um arquivo que você abre no navegador.

package.json e lockfile

O `package.json` registra comandos e dependências diretas. O `package-lock.json` registra as versões resolvidas, inclusive dependências transitivas. Depois que os dois lockfiles estão presentes, `npm ci` na raiz e `npm --prefix backend ci` reproduzem essas instalações sem reescrever o lockfile.

O Express organiza HTTP. Helmet configura cabeçalhos de segurança. Playwright é uma ferramenta de teste e Prettier formata código; nenhum deles é carregado pelo visitante como uma biblioteca do frontend.

04. HTML, DOM e CSS na versão final

O HTML enviado pelo servidor é texto. O navegador o interpreta e constrói a árvore DOM. O CSS afeta a apresentação dessa árvore; o JS consegue selecionar e modificar os objetos que a representam. Essa é a diferença entre editar a fonte no disco e mudar a página carregada.

No projeto, os HTMLs usam elementos semânticos: header, nav, main, section, form, label, input e button. O vínculo `label for="email"` com `id="email"` identifica o campo para leitura assistiva e permite focá-lo pelo rótulo. Já `name="email"` é a chave usada ao coletar o valor com `FormData`.

Um design compartilhado

O início do CSS define tokens: cores, superfícies, bordas e raio. Alterar um token muda os componentes que o usam. Esse é um primeiro passo para um design system: centralizar decisões, em vez de repetir cores diferentes em cada página.

public/css/style.css / linhas 1-25

```
001 /* Shared design tokens keep all four pages consistent without a framework. */
002 :root {
003     color-scheme: dark;
004     --bg: #08121e;
005     --surface: #101e2c;
006     --raised: #142535;
007     --text: #f2f6fa;
008     --muted: #b1c0ce;
009     --accent: #86f0d0;
010     --cyan: #86d8ee;
011     --border: #2a3b4b;
012     --radius: 14px;
013     font-family:
014         Inter,
015         ui-sans-serif,
016         system-ui,
017         -apple-system,
018         "Segoe UI",
019         sans-serif;
020     color: var(--text);
021     background: var(--bg);
022 }
023 * {
024     box-sizing: border-box;
025 }
```

As grades usam Grid e Flexbox para distribuir o conteúdo. Media queries adaptam a navegação e trocam colunas por uma pilha. O layout foi exercitado em 320, 390 e 768 pixels, além da revisão visual em desktop. Responsividade não é só diminuir a fonte: precisa preservar leitura, clique e sequência dos campos.

Acessibilidade faz parte do comportamento

O link "Skip to content" evita atravessar a navegação a cada visita. `:focus-visible` deixa claro onde está o foco do teclado. Mensagens com `role="status"` e `aria-live` podem ser anunciadas sem exigir que o usuário encontre visualmente uma linha que mudou. Os botões de senha mantêm `aria-pressed` junto do estado visual.

Exercício: mude a cor de destaque no token do CSS e observe o efeito em botões, links e foco. Não altere dez seletores individualmente.

05. JavaScript no navegador: módulos e eventos

Cada página carrega um script próprio com `type="module"`. Scripts de módulo têm escopo próprio e são adiados automaticamente durante o processamento normal do documento; não precisam do `defer` usado nos primeiros exercícios. A fonte [2] detalha esse mecanismo.

```
<script type="module" src="/js/register.js"></script>
```

Os arquivos de página importam auxiliares de `api.js`. Essa organização evita copiar tratamento de requisição, erros e estado de formulário em quatro lugares. O backend usa CommonJS (`require` e `.cjs`), enquanto o browser usa módulos ES (`import` e `export`). A linguagem é a mesma; o sistema de carregamento escolhido é diferente.

Eventos e execução adiada

`addEventListener` registra um callback. A função não roda quando você a registra; o browser a chama quando o evento ocorre. `submit` pertence ao formulário e funciona também com Enter. O evento `input` acompanha mudanças de valor; `click` é apropriado para alternar uma visualização.

Promise, async e await

Uma requisição não produz sua resposta imediatamente. `fetch` retorna uma `Promise`. `await` suspende a continuação daquela função assíncrona até o resultado estar disponível; não congela a página inteira esperando a rede. Enquanto isso, outros eventos podem acontecer.

Por isso há um estado ocupado. `bindForm` desabilita o `submit`, marca `aria-busy` e impede uma segunda execução enquanto a primeira aguarda. `finally` restaura a interface, tanto em sucesso quanto em erro.

public/js/api.js / linhas 56-79

```
056 export function bindForm(form, status, handler) {
057   form.addEventListener("input", () => setStatus(status));
058   form.addEventListener("change", () => setStatus(status));
059   form.addEventListener("submit", async (event) => {
060     event.preventDefault();
061     if (!form.reportValidity() || form.getAttribute("aria-busy") === "true")
062       return;
063     const button = form.querySelector('[type="submit"]');
064     const label = button.textContent;
065     button.disabled = true;
066     button.textContent = "Please wait...";
067     form.setAttribute("aria-busy", "true");
068     setStatus(status);
069     try {
070       await handler(new FormData(form));
071     } catch (error) {
072       setStatus(status, error.message, true);
073     } finally {
074       button.disabled = false;
075       button.textContent = label;
076       form.setAttribute("aria-busy", "false");
077     }
078   });
079 }
```

O handler passado a `bindForm` recebe um `FormData` e descreve a operação específica. O auxiliar controla o ciclo de vida comum. Isso é composição de funções, não mágica de framework.

Exercício: use o modo de rede lenta no DevTools e verifique se o botão fica indisponível até a resposta chegar.

06. HTTP: o contrato entre os processos

Uma URL tem protocolo, host, porta e caminho. Em `http://127.0.0.1:3000/api/health`, o caminho é `/api/health`. A raiz `/` é outra rota. O erro inicial "Cannot GET /" aconteceu porque o servidor só conhecia o endpoint de health e ainda não servia o site.

Método e caminho juntos identificam a operação. `GET /api/session` consulta o estado; `POST /api/auth/login` tenta criar uma sessão autenticada. Usar `GET` para sair da conta seria uma escolha ruim: consultas não devem representar ações destrutivas da aplicação.

O contrato implementado

- `GET /api/health`: verifica se a API responde.
- `GET /api/session`: retorna user e csrfToken; pode criar uma sessão anônima.
- `POST /api/auth/register`: recebe nome, email, senha e empresa opcional; cria conta e sessão.
- `POST /api/auth/login`: recebe email e senha; verifica as credenciais.
- `POST /api/auth/logout`: revoga a sessão atual.
- `PATCH /api/profile`: altera nome e empresa da conta autenticada.
- `GET /api/inquiries`: lista somente os pedidos da conta atual.
- `POST /api/contact`: salva um pedido de serviço, com ou sem conta.

Status não é decoração

200 representa sucesso de consulta ou alteração; 201, criação. 400 indica dados inválidos; 401, falta de autenticação ou credenciais incorretas; 403, bloqueio de segurança CSRF/origem; 409, conflito de email; 413, corpo grande demais; 429, limite de requisições; 500, erro interno inesperado.

O backend usa JSON no formato `{ "message": "... " }` para erros da API. Não manda a stack trace ou o hash de senha ao cliente. O frontend pode apresentar a mensagem, mas precisa também respeitar o status.

Exercício: na aba Network, encontre um cadastro. Identifique método, caminho, status, corpo enviado, resposta e cabeçalho Set-Cookie. Faça isso apenas com uma conta de teste.

07. Fetch e o auxiliar api.js

O frontend não chama `createApp` nem abre o arquivo `SQLite`. Ele chama `fetch` para fazer HTTP. O auxiliar `api` encapsula essa tarefa: escolhe headers, serializa JSON, inclui o token CSRF e converte a resposta em objeto.

public/js/api.js / linhas 1-18

```
001 // A cookie identifies the session. The CSRF token stays in this module's
    memory.
002 let csrfToken = "";
003 let sessionPromise;
004
005 export async function api(path, { method = "GET", body } = {}) {
006     const mutation = ![ "GET", "HEAD" ].includes(method);
007     if (mutation && !csrfToken) await getSession();
008     const headers = { Accept: "application/json" };
009     if (body !== undefined) headers["Content-Type"] = "application/json";
010     if (mutation) headers["x-csrf-token"] = csrfToken;
011     let response;
012     try {
013         response = await fetch(path, {
014             method,
015             headers,
016             credentials: "same-origin",
017             ...(body !== undefined ? { body: JSON.stringify(body) } : {}),
018         });
```

`JSON.stringify(body)` transforma o objeto em texto. `Content-Type: application/json` informa ao servidor como interpretar o corpo. `Accept: application/json` informa qual formato de resposta esperamos. `credentials: same-origin` deixa explícito o envio de cookies ao mesmo origin.

Um origin é a combinação de protocolo, host e porta. `127.0.0.1` e `localhost` não são o mesmo host; portas diferentes também criam origins diferentes. Aqui, arquivos e API saem do mesmo servidor, evitando uma configuração CORS desnecessária.

O detalhe de erro que costuma confundir

`Fetch` não rejeita automaticamente sua `Promise` ao receber um HTTP 400, 401 ou 500. Ele recebeu uma resposta válida do protocolo. Precisamos testar `response.ok`, que indica status na faixa 2xx. Erros de conexão são outro caminho, capturado no `catch` ao redor do `Fetch`. Veja [1].

public/js/api.js / linhas 24-38

```
024     const data = await response.json().catch(() => ({}));
025     if (!response.ok) {
026         const error = new Error(
027             data.message ||
028             "The request could not be completed. Please try again.",
029         );
030         error.status = response.status;
031         if (response.status === 403) {
032             csrfToken = "";
033             sessionPromise = undefined;
034         }
035         throw error;
036     }
037     if (data.csrfToken) csrfToken = data.csrfToken;
038     return data;
```

O token CSRF fica na memória do módulo, não no `localStorage`. `getSession` compartilha uma `Promise` para evitar consultas iniciais duplicadas. Ao pedir `refresh`, o `dashboard` consulta novamente o servidor. O cookie de identidade é administrado pelo browser e não é lido pelo JS.

Se uma mutação recebe 403, o auxiliar descarta seu token local. O próximo envio pode obter uma sessão atualizada. Não há repetição automática de POST: reenviar uma ação sem ter certeza do resultado poderia duplicar efeitos.

Exercício: feche o servidor e envie um formulário. Compare a mensagem com a de um login que recebeu 401.

08. O ciclo do cadastro, ponta a ponta

O formulário coleta nome, email, empresa opcional, senha e confirmação. A confirmação é uma regra de interface; não é enviada ao backend. O servidor precisa proteger a senha e criar a conta, mas não precisa guardar uma cópia adicional da mesma informação.

public/js/register.js / linhas 1-29

```
001 import { api, bindForm, setupPasswordToggles } from "../api.js";
002 setupPasswordToggles();
003 const form = document.querySelector("#register-form");
004 const password = document.querySelector("#password");
005 const confirmation = document.querySelector("#confirm-password");
006 function validatePasswordMatch() {
007     confirmation.setCustomValidity(
008         password.value === confirmation.value ? "" : "Passwords do not match.",
009     );
010 }
011 password.addEventListener("input", validatePasswordMatch);
012 confirmation.addEventListener("input", validatePasswordMatch);
013 // Recheck autofilled/programmatically changed fields before the shared handler.
014 form.addEventListener("submit", validatePasswordMatch);
015 bindForm(form, document.querySelector("#register-status"), async (data) => {
016     validatePasswordMatch();
017     if (!form.reportValidity()) return;
018     await api("/api/auth/register", {
019         method: "POST",
020         body: {
021             name: data.get("name").trim(),
022             email: data.get("email").trim(),
023             company: data.get("company").trim(),
024             password: data.get("password"),
025         },
026     });
027     form.reset();
028     window.location.assign("/dashboard.html");
029 });
```

Antes do envio, os eventos `input` revalidam a igualdade. `setCustomValidity` recebe uma mensagem quando o campo é inválido e uma string vazia para remover esse erro personalizado. Isso não desliga `required`, `type=email` ou as restrições de tamanho.

O handler coleta os dados com `FormData`, remove espaços de nome/email/empresa e preserva a senha exatamente como digitada. Depois aguarda a API. Só após sucesso, limpa o formulário e navega para o dashboard. Um erro 409 de email duplicado permanece na página como mensagem; não é tratado como cadastro concluído.

O fluxo no servidor

```
POST /api/auth/register
-> verificar sessao e CSRF
-> validar nome, email, empresa e senha
-> normalizar email e verificar duplicidade
-> calcular hash de senha
-> INSERT parametrizado em users
-> revogar sessao anterior e emitir nova sessao
-> responder 201 com usuario e novo token CSRF
```

A consulta inicial de duplicidade melhora a resposta, mas não substitui o UNIQUE do banco. Dois pedidos simultâneos poderiam passar pelo SELECT antes de qualquer INSERT terminar. A restrição mantém a integridade; o tratamento de erro converte esse conflito em 409.

O objeto público user não contém password_hash. A seleção explícita dos campos evita expor segredos por acidente ao fazer `res.json(row)` com uma linha inteira do banco.

Exercício: tente registrar o mesmo email de novo. Depois tente enviar os dados direto à API sem token CSRF. São falhas diferentes: conflito de regra e bloqueio de segurança.

09. Express: rotas, middlewares e ordem

O Express recebe a requisição e percorre middlewares na ordem em que foram registrados. Um middleware pode responder, chamar `next()` para continuar ou encaminhar um erro. Essa ordem é parte da lógica do programa.

backend/app.cjs / linhas 49-73

```
049     ...options.rateLimit,
050   };
051   app.use("/api", rateLimiter(limits.apiMax, limits.windowMs));
052   app.use("/api/auth", rateLimiter(limits.authMax, limits.windowMs));
053   app.use("/api", (req, res, next) => {
054     res.set("Cache-Control", "no-store");
055     next();
056   });
057   app.use(express.json({ limit: "16kb" }));
058   app.use(session.load);
059   app.use("/api", session.csrf);
060   const userById = (id) =>
061     id
062       ? db
063         .prepare(
064           "SELECT id, name, email, company, created_at AS
065             createdAt FROM users WHERE id = ?",
066         )
067         .get(id) || null
068         : null;
069   app.get("/api/session", (req, res) => {
070     if (!req.session) session.issue(req, res);
071     res.json({
072       user: userById(req.session.user_id),
073       csrfToken: req.session.csrf_token,
074     });
075   });
```

O limite de requisições reduz abuso. A API recebe `Cache-Control: no-store`. `express.json` interpreta o corpo JSON com limite de 16 KB. Em seguida carregamos a sessão; só então verificamos CSRF. Verificar CSRF antes de carregar a sessão impediria comparar o token enviado com o token armazenado.

Depois vêm as rotas, o 404 da API, a proteção do dashboard, os arquivos públicos e o handler de erro. Registrar o diretório estático antes de proteger a página privada permitiria entregar o HTML sem passar pela regra

prevista.

Separação entre app e server

`createApp` monta a aplicação sem escolher uma porta. `server.cjs` lê a configuração e chama `listen`. Nos testes usamos a mesma aplicação com banco temporário e porta automática. Não precisamos editar o programa para alternar entre ambiente real e teste.

backend/server.cjs / linhas 1-22

```
001 "use strict";
002 const { existsSync } = require("node:fs");
003 const path = require("node:path");
004 // Native .env loading preserves inherited environment variables.
005 const envFile = path.join(__dirname, ".env");
006 if (existsSync(envFile)) process.loadEnvFile(envFile);
007 const { createApp } = require("./app.cjs");
008 const port = Number(process.env.PORT || 3000);
009 const host = process.env.HOST || "127.0.0.1";
010 if (!Number.isInteger(port) || port < 0 || port > 65535) {
011   console.error("PORT must be an integer from 0 to 65535.");
012   process.exit(1);
013 }
014 const app = createApp();
015 const server = app.listen(port, host, () => {
016   const displayHost = host.includes(":") ? `[${host}]` : host;
017   console.log(
018     `Server running at http://${displayHost}:${server.address().port}`,
019   );
020 });
021 server.on("error", (error) => {
022   console.error(
```

O arquivo `.env` opcional é carregado nativamente pelo Node. Variáveis herdadas do processo têm precedência. `HOST=127.0.0.1` mantém o serviço local; não significa que ele foi publicado na internet. Ao encerrar, o wrapper fecha o servidor e a conexão do banco.

Exercício: localize a rota de health e acrescente temporariamente um campo `version` na resposta. Veja a mudança no navegador, depois restaure o contrato para não quebrar o teste.

10. Validação: experiência no cliente, autoridade no servidor

A validação HTML permite orientar o visitante cedo. A validação do backend é a regra efetiva, pois o cliente pode ser ignorado. Um programa de teste consegue enviar arrays, null, números ou campos ausentes mesmo que a tela só ofereça inputs de texto.

backend/validation.cjs / linhas 7-39

```
007 function object(body) {
008   if (!body || typeof body !== "object" || Array.isArray(body))
009     invalid("A JSON object is required.");
010   return body;
011 }
012 function text(value, label, min, max, trim = true) {
013   if (typeof value !== "string") invalid(`${label} must be text.`);
014   const result = trim ? value.trim() : value;
015   const length = [...result].length;
016   if (
017     length < min ||
018     length > max ||
019     (trim && /\u0000-\u0008\u000b\u000c\u000e-\u001f\u007f]/.test(result))
020   )
021     invalid(`${label} must contain ${min} to ${max} characters.`);
022   return result;
023 }
024 function email(value) {
025   const normalized = text(value, "Email", 3, 254).toLowerCase();
026   if (!/^[\^s@]+@[^\s@]+\.[^\s@]+$/u.test(normalized))
027     invalid("Enter a valid email address.");
028   return normalized;
029 }
030 function password(value) {
031   return text(value, "Password", 15, 128, false);
032 }
033 function profile(body) {
034   object(body);
035   return {
036     name: text(body.name, "Name", 2, 100),
037     company: text(
038       body.company === undefined ? "" : body.company,
039       "Company",
```

O servidor exige um objeto e depois valida os campos. O helper text verifica tipo, tamanho e caracteres de controle para campos textuais comuns. O tamanho é contado por pontos de código. O navegador e suas restrições de tamanho podem contar unidades UTF-16; portanto, caracteres como emoji merecem atenção se você decidir harmonizar exatamente todos os limites de UI e API.

O email é normalizado com trim e lowercase e submetido a uma verificação sintática simples. Isso não prova que o endereço existe nem que pertence ao usuário. Verificação de email por link seria outra funcionalidade, ainda não implementada.

A senha tem de 15 a 128 caracteres no servidor e não recebe trim. Espaços podem fazer parte de uma passphrase. Não impomos combinações arbitrárias de símbolos ou maiúsculas. Isso é uma política local do projeto, não uma garantia de que qualquer senha aceita será boa.

Campos não são permissões

O cliente não escolhe status do pedido, hash, ID do dono ou data de criação. Esses valores são definidos pelo servidor ou pelo esquema. O perfil aceita nome e empresa; email não pode ser alterado por essa rota. Um futuro fluxo de mudança de email precisaria reautenticação e verificação.

No serviço de contato há uma lista permitida: firewall, pentest, network ou custom. Ter uma lista no select do HTML não basta; a API também precisa aplicar essa regra.

Exercício: adicione uma regra de negócio de empresa com limite menor. Escreva o teste 400 primeiro e depois sincronize o maxlength da interface.

11. SQLite: persistência e relações

SQLite mantém o banco em um arquivo local. Não precisamos de um servidor PostgreSQL separado para este projeto. DatabaseSync abre a conexão e executa suas operações de modo síncrono. Isso simplifica uma aplicação pequena, mas consultas lentas podem bloquear o event loop; não é uma decisão para escalar sem análise. Consulte [4].

backend/database.cjs / linhas 1-25

```
001 "use strict";
002 const { DatabaseSync } = require("node:sqlite");
003 const { mkdirSync } = require("node:fs");
004 const path = require("node:path");
005 function openDatabase(filename) {
006     if (filename !== ":memory:")
007         mkdirSync(path.dirname(filename), { recursive: true });
008     const db = new DatabaseSync(filename);
009     db.exec(`PRAGMA foreign_keys = ON; PRAGMA journal_mode = WAL; PRAGMA
    busy_timeout = 5000;
010     CREATE TABLE IF NOT EXISTS users (
011         id INTEGER PRIMARY KEY, name TEXT NOT NULL, email TEXT NOT NULL UNIQUE
    COLLATE NOCASE,
012         company TEXT NOT NULL DEFAULT '', password_hash TEXT NOT NULL,
    created_at TEXT NOT NULL
013     );
014     CREATE TABLE IF NOT EXISTS inquiries (
015         id INTEGER PRIMARY KEY, user_id INTEGER REFERENCES users(id), name TEXT
    NOT NULL,
016         email TEXT NOT NULL, service TEXT NOT NULL CHECK(service IN
    ('firewall','pentest','network','custom')),
017         message TEXT NOT NULL, status TEXT NOT NULL DEFAULT 'new', created_at
    TEXT NOT NULL
018     );
019     CREATE INDEX IF NOT EXISTS inquiries_owner ON inquiries(user_id, id);

020     CREATE TABLE IF NOT EXISTS sessions (
021         token_hash TEXT PRIMARY KEY, user_id INTEGER REFERENCES users(id) ON
    DELETE CASCADE, csrf_token TEXT NOT NULL, expires_at INTEGER NOT NULL
022     );`);
023     return db;
024 }
025 module.exports = { openDatabase };
```

O significado das tabelas

- **users:** ID, nome, email único, empresa, hash da senha e data de criação.
- **sessions:** digest do token, usuário opcional, token CSRF e expiração.
- **inquiries:** solicitante, serviço, mensagem, status, data e dono opcional.

Uma sessão anônima tem `user_id` nulo. Um contato enviado sem login também pode ter dono nulo. Um pedido feito autenticado recebe o ID da sessão; não passamos esse ID no JSON do browser. Pedidos anônimos não são vinculados retroativamente a uma conta só porque os emails coincidem: email escrito no formulário não comprova identidade.

Integridade e SQL parametrizado

UNIQUE protege o email. Foreign keys descrevem referências entre tabelas. O índice de pedidos por usuário ajuda a consulta do histórico. WAL e busy_timeout são configurações do SQLite para operação local; não transformam o sistema em uma infraestrutura distribuída.

Em uma consulta parametrizada, ? marca valores que serão vinculados separadamente. O texto do visitante não é concatenado ao SQL. Uma apóstrofe num nome não deve quebrar uma instrução nem virar outro comando.

```
// Forma usada no projeto: SQL e valores ficam separados.
db.prepare('SELECT id FROM users WHERE email = ?').get(email);
```

O banco fica por padrão em backend/data/cybershield.sqlite. Não edite esse arquivo com um editor de texto. Não o envie ao Git: contém dados pessoais e material de autenticação. Para backup consistente, considere também a operação em WAL; copiar um arquivo aleatoriamente enquanto há escrita não é uma política de backup.

12. Senhas: hashing não é criptografia reversível

O login precisa verificar se a senha fornecida corresponde à escolhida no cadastro. Não precisa recuperar a senha original. Por isso armazenamos um hash derivado, não texto puro nem uma senha que pode ser descriptografada depois.

backend/passwords.cjs / linhas 1-24

```
001 "use strict";
002 const { scrypt, randomBytes, timingSafeEqual } = require("node:crypto");
003 const { promisify } = require("node:util");
004 const derive = promisify(scrypt);
005 // OWASP's scrypt baseline. Async crypto keeps expensive hashing off the
    event loop.
006 const PARAMETERS = { N: 131072, r: 8, p: 1, maxmem: 256 * 1024 * 1024 };
007 async function hashPassword(password) {
008     const salt = randomBytes(16).toString("hex");
009     const hash = await derive(password, salt, 64, PARAMETERS);
010     return `scrypt$131072$8$1${salt}${hash.toString("hex")}`;
011 }
012 async function verifyPassword(password, stored) {
013     // Missing accounts still perform the expensive operation to reduce
    timing enumeration.
014     const parts = stored?.split("$");
015     const salt = parts ? parts[4] : "00000000000000000000000000000000";
016     const actual = await derive(password, salt, 64, PARAMETERS);
017     const expected = Buffer.from(parts ? parts[5] : "00".repeat(64), "hex");
018     return (
019         expected.length === actual.length &&
020         timingSafeEqual(actual, expected) &&
021         Boolean(stored)
022     );
023 }

024 module.exports = { hashPassword, verifyPassword };
```

O projeto usa scrypt assíncrono do Node, com N=131072, r=8, p=1 e salt aleatório. Esses parâmetros seguem a opção scrypt documentada pela OWASP; Argon2id é sua primeira recomendação geral quando disponível. Não criamos uma função criptográfica própria. Veja [5] e [8].

Por que salt e custo?

O salt distingue hashes de usuários que escolheram a mesma senha e dificulta reaproveitar tabelas pré-computadas. Ele não é um segredo; fica junto do hash. O custo de memória e CPU torna tentativas de adivinhação mais caras, mas também consome recursos do seu servidor.

A derivação é assíncrona para não executar todo o trabalho caro diretamente no event loop. A aplicação limita duas operações de autenticação concorrentes, além de limitar requisições por IP. Não existe segurança em colocar um hash caro e permitir milhares de cálculos paralelos sem controle.

Como a verificação funciona

Ao entrar, o backend recupera o salt do hash armazenado, deriva o valor para a senha apresentada e compara buffers com `timingSafeEqual`. Contas inexistentes ainda percorrem um cálculo caro para reduzir diferença de tempo. A mensagem "Incorrect email or password" não identifica qual dos dois estava errado. No cadastro, o projeto informa conflito de email: essa resposta revela existência e precisa ser reconsiderada conforme o risco de uma implantação pública.

SHA-256 aparece no projeto para tokens aleatórios de sessão, não para senhas humanas. São problemas distintos: tokens têm alta entropia gerada pelo sistema; senhas podem ser previsíveis e exigem derivação resistente a tentativas.

Nunca faça: log de senhas, envio em query string, armazenamento no `localStorage` ou invenção de criptografia caseira. Em produção, HTTPS também é obrigatório: hashing no banco não protege uma senha enviada pela rede em HTTP.

13. Sessão, cookie e logout

HTTP não sabe automaticamente quem fez a requisição anterior. Depois do login, o servidor precisa ligar novas requisições à identidade autenticada. Aqui usamos sessão no servidor e um cookie com token opaco, não JWT.

backend/sessions.cjs / linhas 12-41

```
012 function sessions(db, production, origin) {
013   const cookieOptions = {
014     httpOnly: true,
015     sameSite: "lax",
016     secure: production,
017     path: "/",
018   };
019   const revoke = (req) => {
020     if (req.session)
021       db.prepare("DELETE FROM sessions WHERE token_hash = ?").run(
022         req.session.token_hash,
023       );
024     req.session = null;
025   };
026   return {
027     load(req, res, next) {
028       const token = (req.headers.cookie || "")
029         .split(";")
030         .map((v) => v.trim())
031         .find((v) => v.startsWith(`${COOKIE}=`))
032         ?.slice(COOKIE.length + 1);
033       req.session =
034         token && /^[a-f0-9]{64}$/.test(token)
035         ? db
036           .prepare(
037             "SELECT * FROM sessions WHERE token_hash = ?
AND expires_at > ?",
```

```

038         )
039         .get(digest(token), Date.now())
040         : null;
041     next();

```

O browser recebe Set-Cookie e devolve o cookie em pedidos seguintes compatíveis. HttpOnly impede o JS da página de ler o cookie; SameSite=Lax restringe alguns usos entre sites; Path=/ disponibiliza o cookie nas rotas da aplicação. Secure é habilitado no modo de produção e exige o transporte seguro adequado. Esses atributos não tornam a aplicação invulnerável. Consulte [3] e [6].

Rotação e expiração

backend/sessions.cjs / linhas 43-63

```

043     issue(req, res, userId = null) {
044         revoke(req);
045         db.prepare("DELETE FROM sessions WHERE expires_at <= ?").run(
046             Date.now(),
047         );
048         const token = randomBytes(32).toString("hex");
049         req.session = {
050             token_hash: digest(token),
051             user_id: userId,
052             csrf_token: randomBytes(32).toString("hex"),
053             expires_at: Date.now() + SESSION_MS,
054         };
055         db.prepare("INSERT INTO sessions VALUES (?, ?, ?, ?)").run(
056             req.session.token_hash,
057             userId,
058             req.session.csrf_token,
059             req.session.expires_at,
060         );
061         res.cookie(COOKIE, token, { ...cookieOptions, maxAge: SESSION_MS });
062         return req.session;
063     },

```

O token é aleatório e somente seu digest SHA-256 fica na tabela. No cadastro e login, a sessão anterior é revogada e outra é emitida, com novo CSRF. Isso evita aproveitar a mesma identificação anônima depois da autenticação. A duração configurada é de 24 horas, verificada no servidor; não há extensão automática a cada acesso.

O logout faz duas coisas: apaga a sessão no banco e limpa o cookie. Apenas esconder o dashboard ou apagar um valor no localStorage não encerraria uma sessão no servidor. Um cookie copiado antes do logout também deve falhar depois da revogação.

Quem é o usuário atual?

A cada pedido, `session.load` lê e valida o formato do token, calcula seu digest e consulta uma sessão ainda válida. As rotas protegidas buscam o usuário pelo `user_id` dessa sessão. O frontend nunca apresenta sua própria declaração de identidade como prova suficiente.

Exercício: observe o cookie antes e depois de entrar. O valor deve mudar. Depois faça logout e tente abrir o dashboard novamente.

14. CSRF, XSS e camadas de defesa

CSRF explora o envio automático de cookies para tentar induzir uma ação na sessão da vítima. O projeto cria um token de segurança ligado à sessão e exige esse token no header x-csrf-token das requisições que mudam estado. Verifica também Origin quando presente e rejeita Fetch Metadata cross-site. Veja [7].

backend/sessions.cjs / linhas 64-88

```
064     csrf(req, res, next) {
065         if (["GET", "HEAD", "OPTIONS"].includes(req.method)) return next();
066         const expectedOrigin =
067             origin || `${req.protocol}://${req.get("host")}`;
068         if (
069             (req.get("origin") && req.get("origin") !== expectedOrigin) ||
070             req.get("sec-fetch-site") === "cross-site" ||
071             !req.session ||
072             !equal(req.get("x-csrf-token"), req.session.csrf_token)
073         ) {
074             return res
075                 .status(403)
076                 .json({
077                     message:
078                         "Invalid security token or origin. Refresh the
page and try again.",
079                 });
080         }
081         next();
082     },
083     logout(req, res) {
084         revoke(req);
085         res.clearCookie(COOKIE, cookieOptions);
086     },
087 };
088 }
```

A sessão anônima existe também para proteger cadastro, login e contato antes de haver usuário autenticado. O token CSRF não é a senha nem o cookie de identidade. O frontend o obtém pelo endpoint de sessão e o mantém em memória.

XSS é outro problema

XSS ocorre quando um conteúdo não confiável vira código executável na página. O histórico de pedidos contém texto escrito pelo visitante. A renderização cria elementos e atribui textContent; não injeta mensagens com innerHTML. Um texto contendo uma tag deve aparecer como texto, não criar um elemento com evento.

public/js/dashboard.js / linhas 46-67

```
046     for (const inquiry of inquiries) {
047         const article = document.createElement("article");
048         article.className = "inquiry";
049         const top = document.createElement("div");
050         top.className = "inquiry-top";
051         const title = document.createElement("h3");
052         title.textContent = services[inquiry.service] || "Service request";
053         const badge = document.createElement("span");
054         badge.className = "badge";
055         badge.textContent = `Status: ${inquiry.status}`;
056         top.append(title, badge);
057         const time = document.createElement("time");
058         time.textContent = formatDate(inquiry.createdAt);
059         time.dateTime = inquiry.createdAt;
060         const message = document.createElement("p");
061         message.className = "inquiry-message";
062         message.textContent = inquiry.message;
063         const meta = document.createElement("p");
```

```

064     meta.className = "inquiry-meta";
065     meta.textContent = `${inquiry.name} · ${inquiry.email}`;
066     article.append(top, time, message, meta);
067     list.append(article);

```

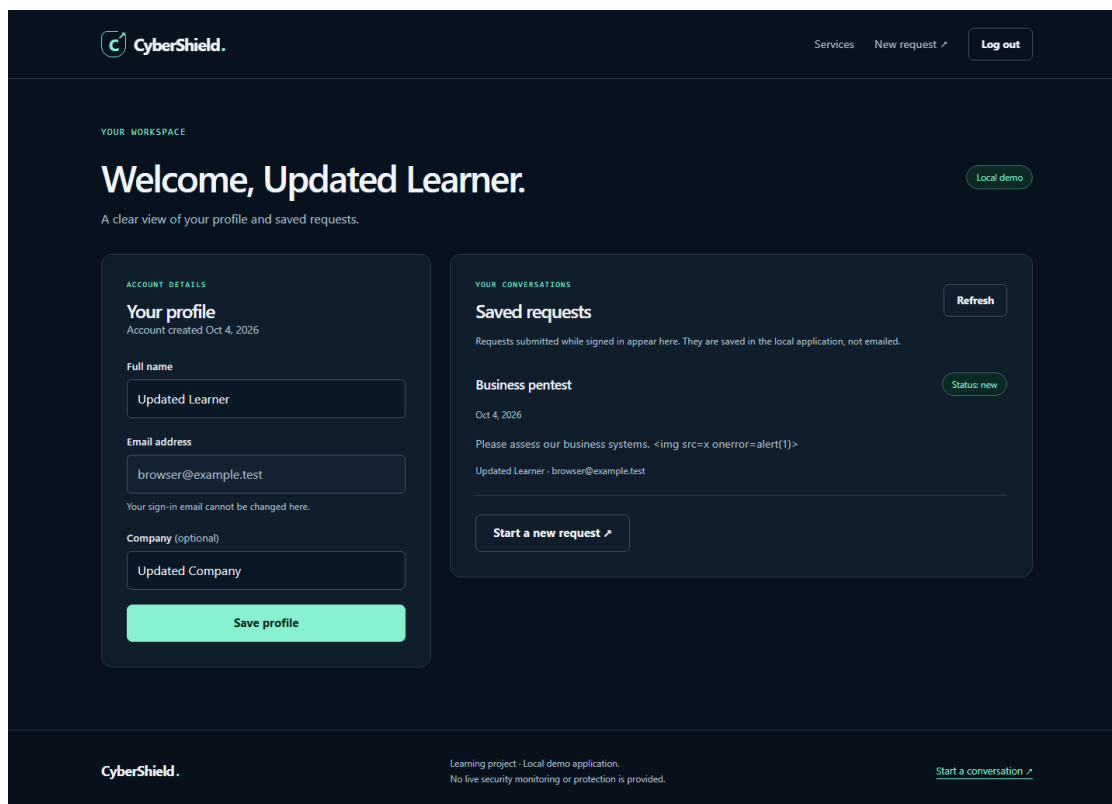
Helmet envia uma Content Security Policy limitada ao próprio origin. Os scripts são externos, os estilos são locais e não precisamos permitir execução inline. CSP é defesa adicional, não justificativa para deixar de tratar dados como texto.

Outros limites reais

- SQL parametrizado evita misturar texto recebido com instruções SQL.
- Limite de corpo restringe o tamanho de JSON aceito.
- Rate limiting e limite de hashing controlam parte do consumo de recursos.
- no-store é usado na API e no dashboard para reduzir armazenamento indevido de conteúdo privado.
- Erros 500 devolvem mensagem genérica, não detalhes internos.

Nenhum conjunto desses controles substitui revisão de segurança e operação correta. XSS pode agir dentro da mesma origem; HttpOnly não impede um script malicioso de fazer requisições em nome do usuário. CORS, CSRF, autenticação e autorização são conceitos diferentes.

15. Dashboard, perfil e pedidos de serviço



O dashboard não é uma tela de monitoramento fictícia. Ele reúne dados reais: nome, email, empresa, data da conta e pedidos registrados enquanto o usuário estava autenticado. O estado vazio é explícito quando ainda não existe pedido.

Proteção em duas camadas

O servidor verifica a sessão antes de entregar `dashboard.html`. O JS também consulta a sessão e redireciona se ela expirou. Essa segunda verificação melhora a experiência, mas não substitui as rotas de API protegidas. Mesmo que alguém obtenha uma cópia do HTML, não deve conseguir consultar o histórico sem credenciais.

No Windows, nomes de arquivos podem ignorar maiúsculas e alguns sufixos. Por isso a proteção normaliza o caminho antes do middleware estático. Os testes cobrem variações do caminho da página, não só sua grafia mais óbvia.

Autorização por propriedade

backend/app.cjs / linhas 166-189

```

166     const authenticated = (req, res, next) => {
167       req.user = userById(req.session?.user_id);
168       if (!req.user)
169         return res
170           .status(401)
171           .json({ message: "Please log in to continue." });
172       next();
173     };
174     app.patch("/api/profile", authenticated, (req, res) => {
175       const profile = validate.profile(req.body);
176       db.prepare("UPDATE users SET name = ?, company = ? WHERE id = ?").run(
177         profile.name,
178         profile.company,
179         req.user.id,
180       );
181       res.json({ user: userById(req.user.id) });
182     });
183     app.get("/api/inquiries", authenticated, (req, res) => {
184       const inquiries = db
185         .prepare(
186           "SELECT id, name, email, service, message, status, created_at
187           AS createdAt FROM inquiries WHERE user_id = ? ORDER BY id DESC",
188         )
189         .all(req.user.id);
190       res.json({ inquiries });

```

A consulta do histórico usa `WHERE user_id = ?` com o ID obtido na sessão. A alteração de perfil usa o mesmo princípio. Não basta perguntar “está logado?”: é preciso perguntar “pode acessar este recurso?”. Esse é o começo da autorização.

O contato salva um pedido com `status=new`. Não há mudança de status na interface nem painel administrativo. O dashboard oferece Refresh para buscar novamente; não existe atualização por WebSocket. Um pedido anônimo fica no banco sem aparecer para outras contas.

O frontend usa `Intl.DateTimeFormat` para apresentar datas em inglês. A data original ISO continua vindo do servidor. Apresentação é responsabilidade do browser; o instante persistido é responsabilidade do backend.

Exercício: com duas contas de teste, crie um pedido na primeira e verifique que a segunda recebe lista vazia. Não implemente isso filtrando apenas no frontend.

16. Testes: observar o comportamento, não só a sintaxe

`node --check` encontra erros de sintaxe, mas não garante que o script carregou, que o seletor existe ou que o formulário enviou a requisição correta. O projeto usa camadas complementares de testes.

```
npm test
npx playwright install chromium
npm run test:e2e
```

A execução verificada passou em **15 testes de backend**, **4 testes do frontend** e **30 verificações no navegador**. O relatório de browser fica em `docs/e2e-results.json`, e as capturas em `docs/screenshots/`. São números da execução deste projeto, não uma promessa de cobrir todos os casos possíveis.

O que cada camada cobre

Os testes HTTP criam uma aplicação real, usam SQLite temporário e verificam cadastro, login, CSRF, origem, perfil, propriedade dos pedidos, persistência, limites, headers e acesso aos arquivos. A persistência é testada recriando a aplicação sobre o mesmo banco de teste.

Os testes do frontend verificam o contrato do auxiliar de requisições e características essenciais das páginas. O browser executa o caminho completo: cadastro com Enter, dashboard, edição, contato, logout, senha incorreta, login correto e tamanhos de tela. Também verifica que um trecho com aparência de HTML permanece texto.

Por que um teste quebrou durante a formatação?

Um teste procurava dois atributos separados por um único espaço. Prettier colocou uma quebra de linha entre eles, sem alterar o comportamento do HTML. Ajustamos a verificação para localizar os atributos no mesmo input sem depender da formatação. Um teste bom protege o contrato; não deve exigir uma apresentação irrelevante da fonte.

Os testes usam credenciais sintéticas e bancos temporários. Não inserem pessoas de exemplo no banco da aplicação que você vai usar. Depois de rodar, limpam o banco de teste. O script E2E abre sua própria porta livre, sem depender de um servidor já iniciado por você.

Exercício: escreva primeiro um teste que falhe para um serviço desconhecido no contato. Depois identifique qual regra já satisfaz ou precisa satisfazer essa expectativa.

17. DevTools e investigação de problemas

Use uma investigação por camadas: primeiro a requisição, depois a resposta, depois a interpretação da interface. Não mude CSS para corrigir um POST 401 nem acrescente um try/catch para esconder uma regra de autorização quebrada.

Network

Abra F12, aba Network, filtre por Fetch/XHR e envie o formulário. Veja URL, método, status e JSON. Um 403 costuma significar sessão/token/origem incompatíveis; 400, validação; 409, conflito; 429, limite; 500, problema interno. Não cole senhas reais, cookies ou tokens em mensagens de suporte.

Console e Sources

Console mostra falhas de carregamento e erros de JS. Um `null.addEventListener` geralmente aponta para seletor errado, script na página errada ou momento de execução. Sources permite pausar no handler do formulário e inspecionar valores. Não confunda encontrar um erro com apenas silenciá-lo.

Application e cookies

Inspeione atributos e expiração do cookie. O fato de o cookie `HttpOnly` aparecer no DevTools não significa que seu JavaScript consiga lê-lo. DevTools é ferramenta do dono do navegador, não uma permissão da página.

Problemas comuns deste projeto

- "Cannot GET /": servidor antigo sem configuração estática ou URL errada.
- Porta ocupada: outro processo já escuta em 3000; encerre o correto ou configure PORT.
- Abrir HTML por `file://`: use o servidor, pois módulos e API usam caminhos do origin.
- npm não encontrado: instalação/PATH do seu terminal; reabra após instalar Node.
- Sessão não permanece: misturar localhost e 127.0.0.1, usar cookies Secure sem HTTPS ou bloquear cookies.
- Dados não aparecem: pedido foi anônimo ou outra conta está autenticada.

Configuração local

Copie `backend/.env.example` para `backend/.env` apenas se precisar mudar defaults. Não é obrigatório para começar. O diretório `data/` é criado no primeiro uso. Um `DB_PATH` relativo é resolvido a partir do diretório de trabalho do processo; prefira caminho absoluto quando configurar manualmente.

18. Limites antes de publicar e próximos passos

Uma aplicação funcionando localmente não é uma implantação de produção. O projeto tem controles básicos testados, mas ainda não tem verificação de email, recuperação de senha, MFA, painel de atendimento, entrega de email, observabilidade operacional ou políticas completas de retenção de dados.

Checklist de publicação

- HTTPS, domínio e origin explícito corretamente configurados.
- Revisão do proxy reverso; não ativar trust proxy sem entender os hops confiáveis.
- Cookies Secure, restrições de origem e rate limiting funcionando nesse ambiente.
- Backup consistente do banco e teste de restauração.
- Atualizações, acompanhamento de vulnerabilidades e plano de resposta a falhas.
- Email de verificação e recuperação de senha com tokens expirados e de uso único.
- Proteção contra abuso, cadastro automatizado e tentativas distribuídas.
- Regras de acesso para eventual painel administrativo, nunca baseadas só no frontend.
- Política de privacidade, consentimento e retenção adequada aos dados tratados.

O rate limiter atual usa memória do processo: reiniciar zera a contagem, e várias instâncias não compartilham o limite. O SQLite síncrono serve ao estudo local; escalabilidade, backup e concorrência precisam ser avaliados para uma implantação real. Não basta trocar HOST por 0.0.0.0 e chamar isso de produção.

Uma sequência de evolução que ensina

Primeiro acrescente testes de uma nova regra. Depois implemente um filtro de pedidos no servidor. Em seguida planeje alteração de senha com reautenticação e revogação de outras sessões. Só então explore componentes React usando a mesma API. TypeScript pode tornar contratos mais claros, mas tipos estáticos não validam automaticamente JSON recebido de um cliente.

Desafio final: desenhe o cadastro de memória e marque, para cada etapa, processo, formato dos dados, ponto de validação e efeito persistido. Se você consegue fazer isso, já entende a lógica central do projeto.

19. Exercícios com critérios e pistas

A. Melhorar a experiência do formulário

Mostre uma contagem de caracteres da mensagem de contato. Critério: atualiza ao digitar ou colar, não altera a mensagem e não interfere no submit. Pista: escute `input` no `textarea`; atualize um elemento de texto; lembre que a validação do servidor continua necessária.

B. Acrescentar uma regra de negócio

Defina um limite menor para a empresa e faça a API rejeitar excessos com 400. Critério: valores válidos persistem; valores inválidos não alteram o perfil; cliente e servidor têm limites compatíveis. Pista: comece em `validation.cjs` e escreva a expectativa antes da mudança.

C. Filtrar pedidos por serviço

Adicione um filtro opcional no GET do histórico. Critério: a conta continua vendo só seus próprios pedidos; serviço inválido é rejeitado; sem filtro mantém o comportamento atual. Pista: o `WHERE` de proprietário nunca deve desaparecer ao acrescentar outra condição.

D. Testar revogação

Use uma sessão de teste, faça logout e tente a API com o cookie antigo. Critério: a consulta privada retorna 401. Pista: o teste precisa preservar o cookie para a tentativa negativa; só observar o browser na tela de login não prova revogação no banco.

E. Planejar uma mudança de senha

Antes de codificar, liste as rotas e os controles: senha atual, nova política, CSRF, hash, limitação de tentativas e revogação das demais sessões. Critério: o fluxo não permite mudar a senha só porque alguém sabe o email. Pista: autenticação existente não dispensa avaliar reautenticação para uma operação sensível.

F. Migrar um componente para React

Reimplemente a lista de pedidos sem mudar o backend. Critério: os mesmos testes de propriedade continuam passando; conteúdo não confiável continua texto; estados vazio, carregando e erro permanecem acessíveis. Pista: compare estado e renderização de componentes com `createElement/replaceChildren`, em vez de reaprender HTTP do zero.

Como conferir aprendizado

Para cada exercício, escreva uma hipótese, uma alteração pequena e uma verificação observável. Procure explicar por que o código funciona. Copiar uma solução que passa uma vez não é o mesmo que conseguir manter o sistema quando a resposta da API falha ou outro usuário faz a mesma ação.

20. Referências e mapa final de leitura

As fontes abaixo são documentação primária e guias de segurança. As explicações de fluxo e os exemplos deste manual descrevem o código implementado. Para decisões futuras, leia a documentação da versão que você executar e não trate exemplos de estudo como políticas universais.

- [1] MDN, Using the Fetch API: Promises, response.ok e leitura de respostas.
- [2] MDN, JavaScript modules: import/export e carregamento de módulos.
- [3] MDN, Set-Cookie: HttpOnly, Secure, Path e SameSite.
- [4] Node.js, SQLite: DatabaseSync, statements e parâmetros.
- [5] Node.js, Crypto: script, randomBytes e timingSafeEqual.
- [6] OWASP, Session Management Cheat Sheet: ciclo de vida e proteção de sessões.
- [7] OWASP, Cross-Site Request Forgery Prevention Cheat Sheet: tokens e contexto da requisição.
- [8] OWASP, Password Storage Cheat Sheet: funções de derivação e configurações de custo.
- [9] Express, Security Best Practices: orientações para implantação.

[1] https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
[2] <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>
[3] <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie>
[4] <https://nodejs.org/api/sqlite.html>
[5] <https://nodejs.org/api/crypto.html>
[6] https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
[7] https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
[8] https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
[9] <https://expressjs.com/en/advanced/best-practice-security.html>

Ordem recomendada dos arquivos

Leia `public/js/register.js`, depois `public/js/api.js`. Acompanhe `backend/app.cjs` e seus módulos de validação, senha e sessão. Observe `database.cjs` para entender a persistência. Volte ao `dashboard.js` e compare o que ele mostra com o que a API autoriza. Finalmente leia os testes: eles documentam comportamentos esperados de forma executável.

O PDF é regenerável a partir deste Markdown e de `docs/build_guide.py`. Os exemplos @code são extraídos da fonte local com numeração de linhas, por isso uma mudança nos arquivos exige revisar os intervalos e reconstruir o guia. A fotografia preservada no site é creditada a Kevin Ache via Unsplash no README do projeto.